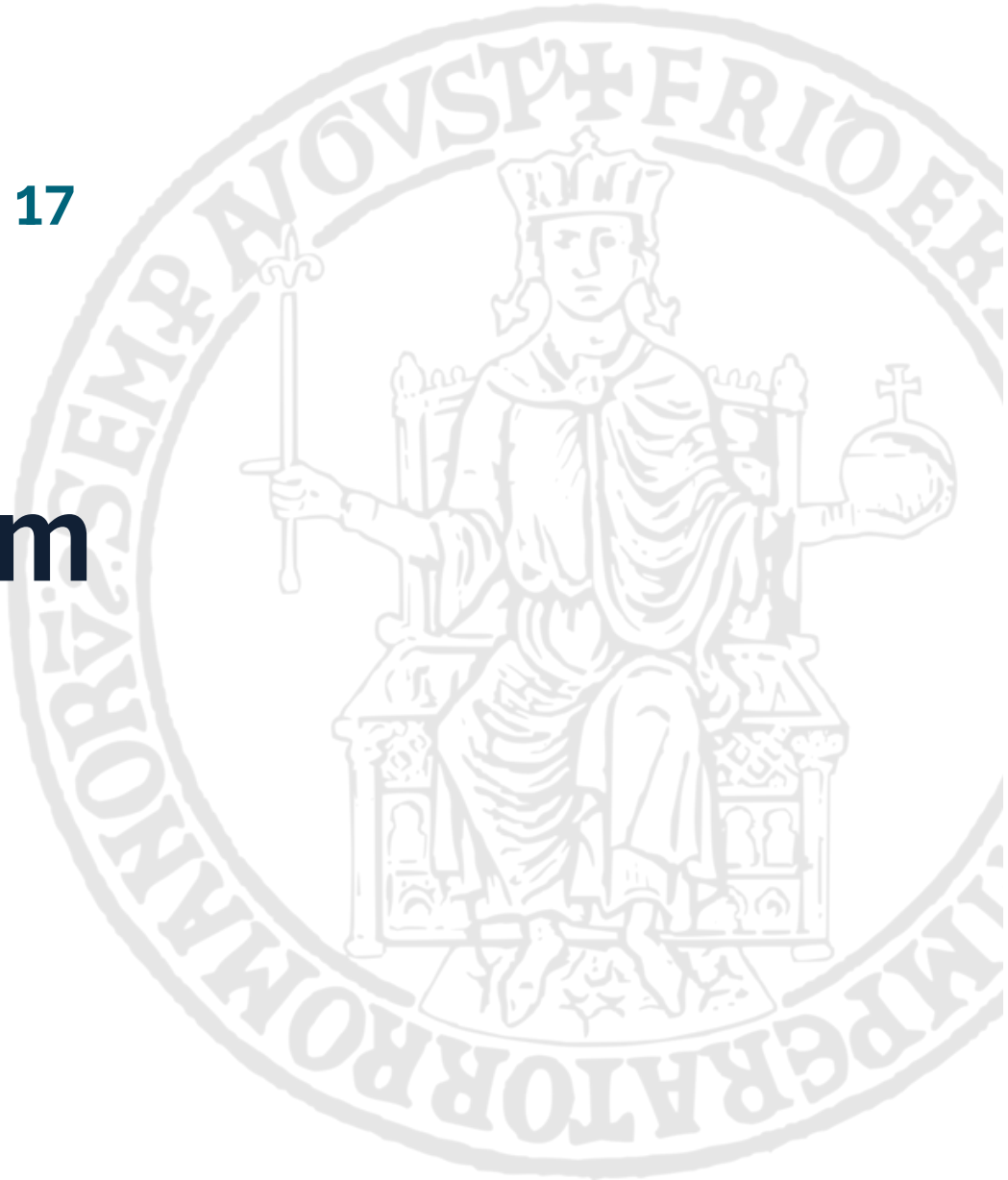
UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SOFTWARE ENGINEERING – LECTURE 17

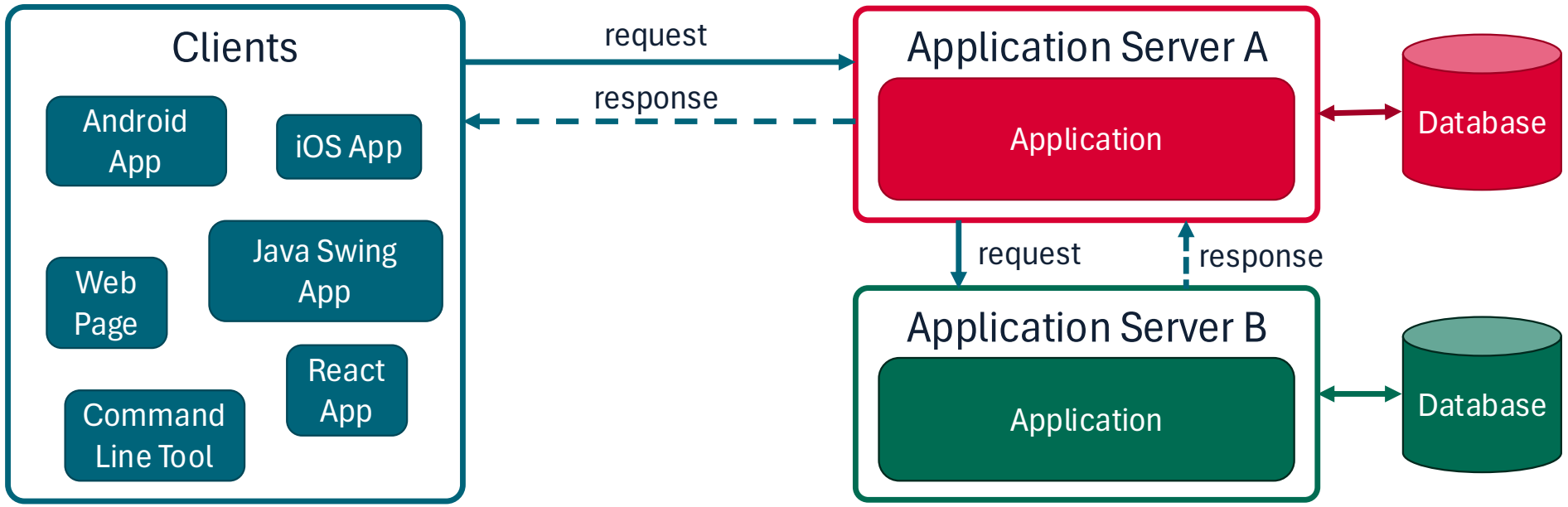
The REST Paradigm

Prof. Bernardo Breve



Architectural Context

- **Software** needs to communicate over the **Internet**
- **83% of web traffic is actually API calls**[1]

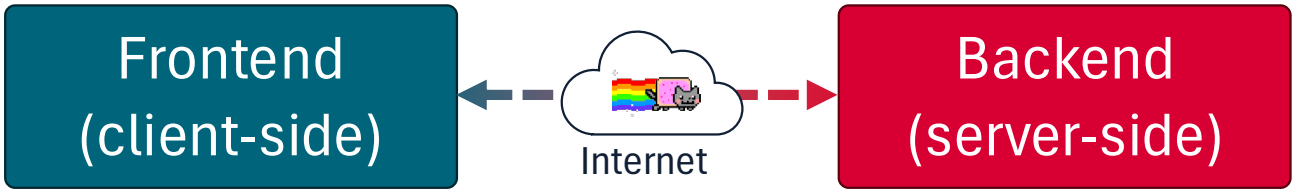


[1] Akamai's State of the Internet Security Report (2019)
https://www.akamai.com/newsroom/press-release/state-of-the-internet-security-retail-attacks-and-api-traffic

Client-Server Architectures

Often, applications are **split** in two components:

- **Backend**: Responsible for data and business logic (on the server-side)
- **Frontend**: Responsible for the UI (on the client-side)
  - Might be a **native mobile** or **desktop app**
  - Or it might still be a «*smarter*» HTML page, that renders an interactive UI leveraging JavaScript (i.e., Single Page Web Apps)
- These two components communicate over the internet



Application Programming Interfaces

Application Programming Interfaces (**API**s) are ways for computer programs to communicate with each other

How can we handle communications over the internet?

- Manually define a protocol over TCP/UDP, open sockets, etc…
  - Not very cost-effective, not very **interoperable**
  - If we want to integrate *n* APIs, we'll need to learn *n* different protocols
- CORBA, Java RMI, SOAP, …
  - Dedicated protocols/architectures exist(ed), re-inventing an alternative to the web
- Just use **web protocols (HTTP)**!

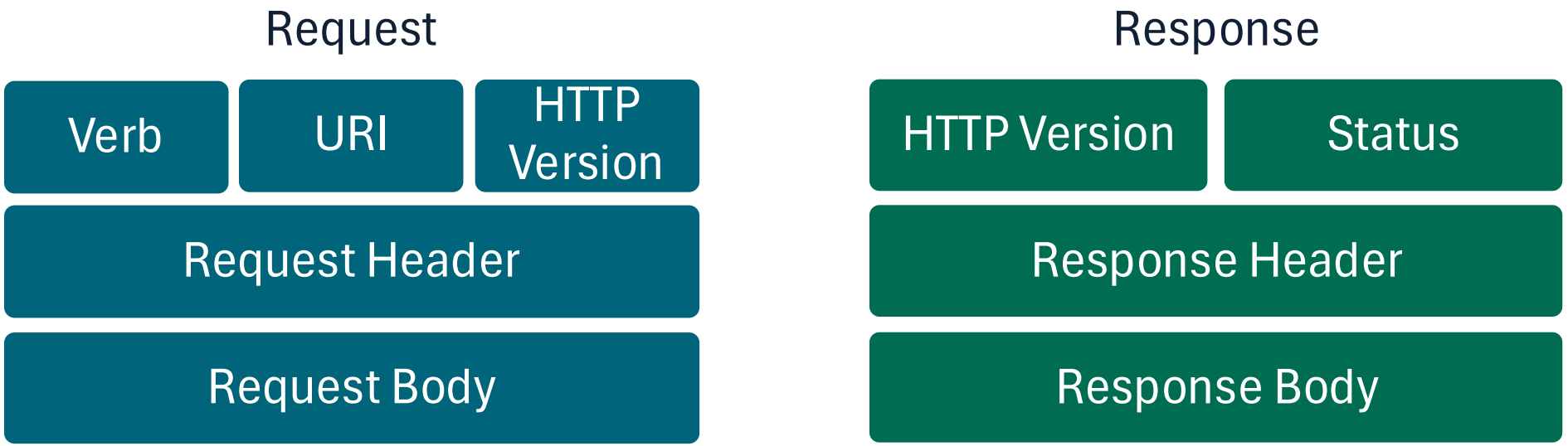
REST

- REST is **not** a standard or a protocol
- REST is an **architectural style**: a set of principles and guidelines that define how web standards should be used in APIs
- Based on HTTP and URIs
- Provides a common and consistent interface based on «proper» use of HTTP
- The prevalent web API architecture style with a **93.4% adoption rate**[1]

[1] https://nordicapis.com/20-impressive-api-economy-statistics/

(A look back on) HTTP

- HyperText Transfer Protocol
- Two types of messages: Request and Response



HTTP Requests

- HTTP can request different kinds of operations (using **Verbs**)
- The resource on which to operate is identified by the URI
- Based on idea of CRUD (Create, Retrieve, Update, Delete)

Verb	Operation Description
POST	Here is some new data. Save it and **CREATE** a new resource
GET	**RETRIEVE** information about the resource
PUT	Here is **UPDATE**d info about the resource
DELETE	**DELETE** this resource

HTTP Responses

- Status codes give information about the outcome of the request

Status Code	Meaning
200	Request was successfully handled
201	A resource was successfully created
400	Cannot understand the request
401	Authentication failed, or not authorized.
404	Resource not found
405	Method not supported by resource
500	Application error

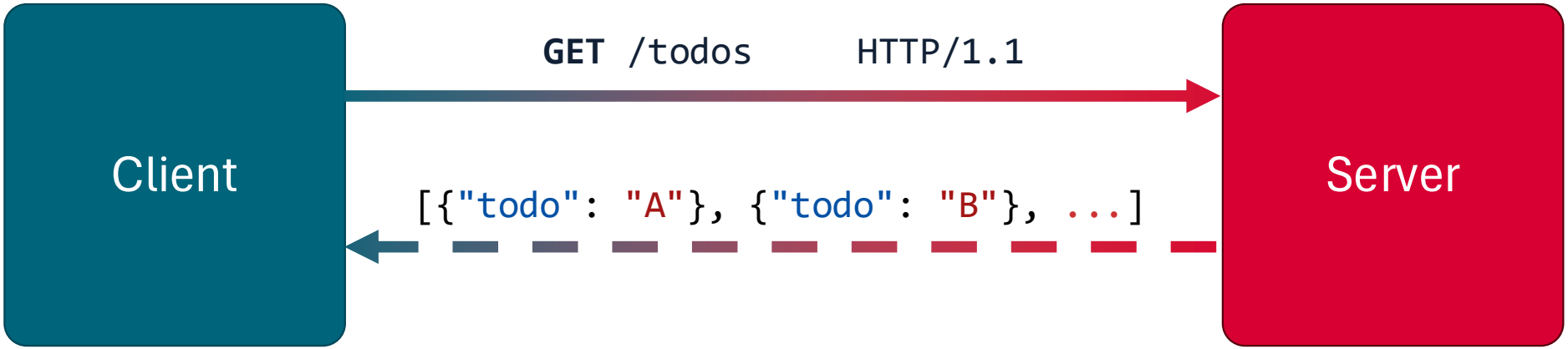
REST Fundamentals

- A REST API allows to interact with **resources** using HTTP
- All resources are associated to a unique URI
- «A resource is anything that's important enough to be referenced as a thing in itself.»[1]
- Resource typically (but not necessarily) correspond to persistent domain objects
- HTTP verbs should be used to retrieve or manipulate resources
- REST API should be **stateless** (e.g.: not use server-side sessions)
  - This ensures better scalabilty! Can you tell why?

[1] Richardson, Leonard, and Sam Ruby. *RESTful web services*. "O'Reilly Media, Inc.", 2008.

Representational State Transfer (REST)

- **Application State** (on the Client)
- **Resource State** (on the Server)
- Trasferred using appropriate representations (e.g.: JSON, XML, ...)



HTTP: Data Interexchange Formats

- Widely used formats include JSON and XML

```
{
  "todos": [{
    "todo": "Learn REST",
    "done": false
  }, {
    "todo": "Learn JavaScript",
    "done": true
  }]
}
```

```
<?xml version="1.0" encoding="UTF-8" ?>
<root>
  <todos>
    <todo>
      <text>Learn REST</text>
      <done>false</done>
    </todo>
    <todo>
      <text>Learn JavaScript</text>
      <done>true</done>
    </todo>
  </todos>
</root>
```

REST: Examples

HTTP verb/URI	Meaning
GET /todos	Retrieve a list of all saved To-do items
GET /todos/<ID>	Retrieve only the To-do item whose ID is <ID>
POST /todos	Save a new To-do item. Data of the exam to save are in the request body
PUT /todos/<ID>	Replace (or create) the To-do item whose ID is <ID>, using the data in the request body
DELETE /todos/<ID>	Delete the To-do item having ID <ID>

REST: Nested Resources

- Sometimes, there is a hierarchy among resources
  - A Company has 0..* Departments which have 0..* Employees…
- When designing an API, it is possible to define nested resources
- **`GET /listings/{id}/reviews`** lists all the reviews of a given listing.
- Keep in mind that every resource should have only one URI

REST: Filtering and Ordering

- Often, clients need to retrieve a list of resource with specific filtering criteria or specific sorting in place (e.g.: sort listings by price)
- You may be tempted to add new paths as follows

HTTP verb/URI	Meaning
GET /todos	Retrieve a list of all saved To-do items
GET /todosSortedAsc	Retrieve saved To-do items, sorted alphabetically (ascending order)
GET /todos/filter/key	Retrieve all To-do Items containing the «key» keyword

REST: Filtering and Ordering

HTTP verb/URI	Meaning
GET /todos	Retrieve a list of all saved To-do items
GET /todosSortedAsc	Retrieve saved To-do items, sorted alphabetically (ascending order)
GET /todos/filter/key	Retrieve all To-do Items containing the «key» keyword

- This is not REST compliant!
  - Resources should be accessed via a single path!
- Also, this can lead to complex and hard-to-use APIs
  - Think of all possible sorting and filtering options you may need

REST: Filtering and Ordering

- So, how do we handle these cases?
- You should not add new paths, but you should use **query parameters**

HTTP verb/URI	Meaning
GET /todos	Retrieve a list of all saved To-do items
GET /todos?sort=title&mode=asc	Retrieve saved To-do items, sorted alphabetically by title (ascending order)
GET /todos?q=key	Retrieve all To-do Items containing the «key» keyword

Securing a REST API

The JSON Web Token (JWT) Authentication/Authorization Scheme

API Authentication/Authorization

- REST APIs allow users to manipulate resources
- In most cases, we don’t want everyone to be able to do so
- **Authentication**: We want that only legit users can access the resources
- **Authorization**: We may also want that some users can access only certain resources (e.g.: an employee shouldn’t be able to update its own salary)

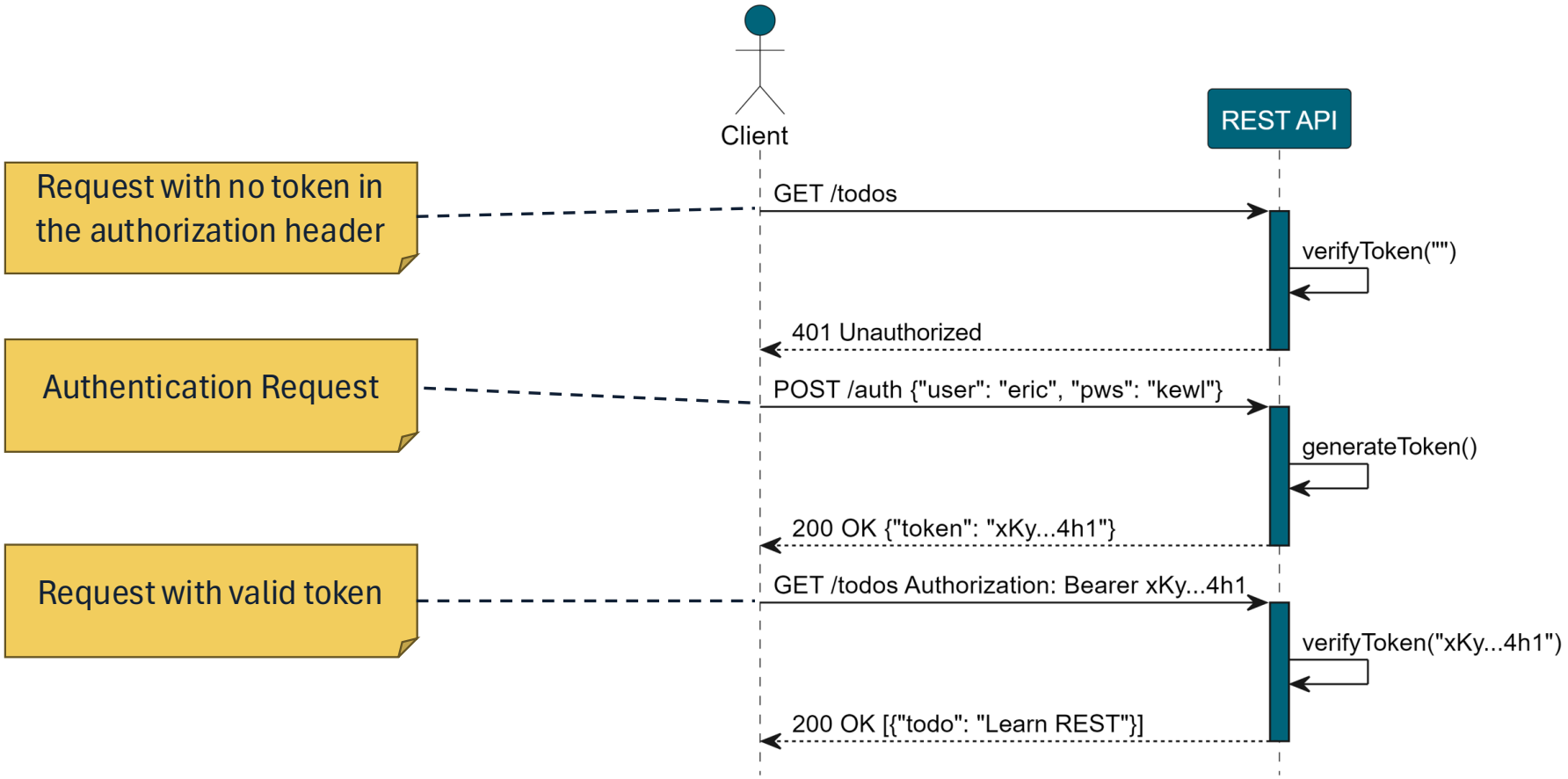
How to secure REST APIs

A widely-used authentication scheme is based on Tokens

💡 Idea:

1. Clients send a request with username and password to the API
2. API validates username and password, and generates a token
3. The token (a string) is returned to the client
4. Client must pass the token back to the API at every subsequent request that requires authentication (in the Authorization Header)
5. API verifies the token before responding

Token-based Authentication Scheme



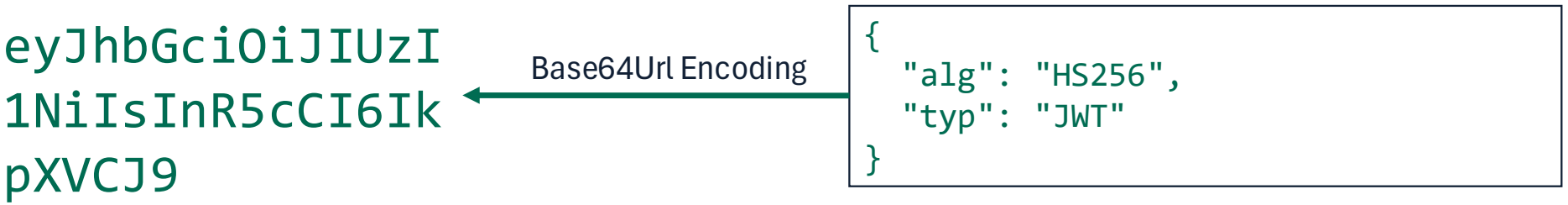
JSON Web Token (JWT)

- JWT is a widely-adopted open standard (RFC 7519)
- Allows to securely share claims between two parties
- A JWT token is a string consisting of three parts, separated by “.”
- Structure: Header.Payload.Signature

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpX
VCJ9.eyJuYW1lIjoibHVpZ2kiLCJyb2x
lIjoiYWRtaW4iLCJleHAiOjE2NzAzOTg
0MzJ9.fopBYrax8wcB7rnPjCcOMc62IT
2lJdvyOdyixMWMZAQ
```

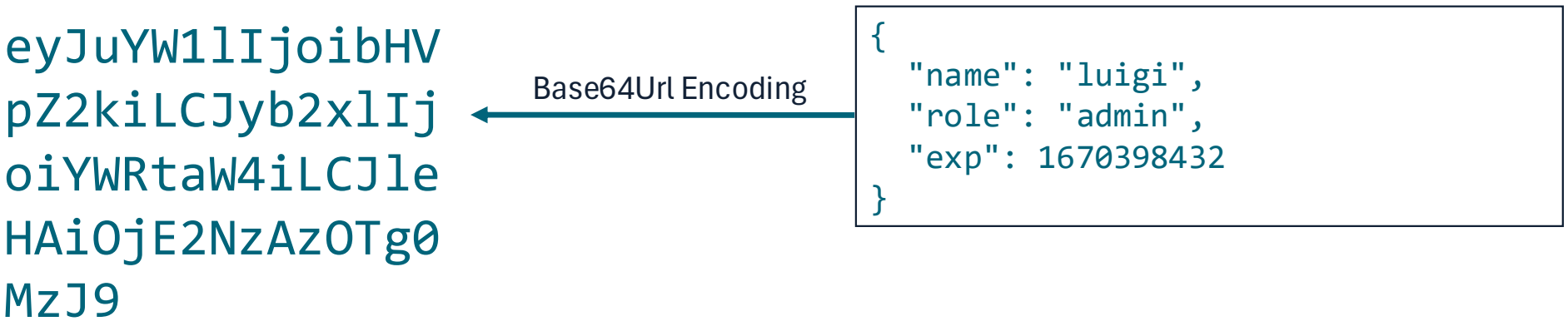
JWT: Header

- The JWT header contains information about the **type of token** and the type of hashing function used to **sign it**, in JSON format
- It is **encoded** using Base64Url Encoding
  - Note that this is merely an encoding, not an encryption! **It can be easily be inverted!**



JWT: Payload

- The payload is a JSON object containing **claims**
- Some registered claims are recommended (e.g. «exp» indicates an expiration date for the token)
- Claims are customizable (we can write any claim in the payload)
- It is **encoded** using Base64Url Encoding



JWT: Signature

- To ensure that tokens are not tampered or forged, a signature is included
- The signature is obtained by applying a **cryptographic hashing function** to the string obtained by concatenating:
  - The **header** of the token
  - The **payload** of the token
  - A **secret key** known only by the server that issues the token
- Actually, JWT signatures are a little bit more complex than that
  - Check out HS256 or RS256 if you want to know more: https://auth0.com/blog/rs256-vs-hs256-whats-the-difference/

Cryptographic Hashing Functions

Functions that map an arbitrary binary string (**input**) to a fixed-size binary string (**digest** or **hash**)

Secure cryptographic hashing funcs have some interesting properties:

- They are virtually **collision free** (basically it's impossible to find two different inputs that lead to the same digest)
- They can be computed easily, but **cannot be inverted**
  - Given an input, it is easy to compute its digest. But given a digest, it's unfeasible to compute the input that generated it

JSON Web Token: Overview